

UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

DEPARTMENT OF INTERNET OF THINGS AND ROBOTICS ENGINEERING (IRE)

FACULTY OF CYBER PHYSICAL SYSTEM ENGINEERING

LAB REPORT 01

OBJECT-ORIENTED DESIGN IN DART: DIGITAL WALLET BACKEND SYSTEM

(Encapsulation, Inheritance, and Polymorphic Method Overriding)

COURSE CODE: IRE 460

COURSE TITLE: MOBILE PLATFORM FOR IOT DEVICES SESSIONAL

SUBMITTED BY:

Shahriar Alom Masud

Student ID: **2201032** | Session: 2022–2023

Department of Internet of Things and Robotics Engineering (IRE), UFTB

SUBMITTED TO:

Md. Rafiqul Islam

Lecturer

Department of Internet of Things and Robotics Engineering (IRE)

University of Frontier Technology, Bangladesh

NOVEMBER 03, 2026

1. Problem Statement and Architectural Requirements

Modern digital wallets require secure, modular, and maintainable financial transaction backends. In this laboratory task, a backend system is designed for a Digital Wallet application that manages two categories of accounts: **Standard Accounts** and **Premium Accounts**. The core business logic mandates rewarding premium users with an automatic cash-back bonus on every monetary deposit.

Architectural Specifications:

1. **Encapsulation (`BankAccount` Base Class):**
   - Create a fundamental base class named `BankAccount`.
   - Declare the balance attribute (`_balance`) as **private** using Dart's privacy convention (leading underscore).
   - Expose a public read-only getter (`double get balance => _balance;`) to inspect account liquidity while strictly preventing unauthorized direct modification from outside the class.
   - Implement a robust `deposit(double amount)` method to validate and credit deposits.
2. **Inheritance and Polymorphism (`PremiumAccount` Derived Class):**
   - Create a derived class named `PremiumAccount` that extends `BankAccount`.
   - Override the `deposit(double amount)` method to inject rewarding logic: premium accounts automatically receive an additional **5% cash-back bonus** added to their deposit balance.
   - Maintain clean delegation by delegating internal balance mutations through the superclass constructor and method interfaces.

GitHub Repository Link:
https://github.com/Masud744/flutter-lab/tree/main/Lab_01

2. Object-Oriented Principles and System Architecture

2.1 Architectural Pillars

- **Encapsulation (Information Hiding):** In Dart, encapsulation is enforced at the library boundary using the underscore prefix (_). By encapsulating `_balance`, arbitrary external mutations such as `account._balance = 999999;` are prohibited at compile time. Read access is mediated via the getter `balance`, while write access is constrained exclusively through the validated `deposit()` interface.
- **Inheritance (Code Reuse & Specialization):** The `PremiumAccount` class inherits all structural attributes and behavioral methods from `BankAccount` through the `extends` keyword. The sub-class constructor utilizes super-initializer parameters (`PremiumAccount([super.initialBalance])`) to forward configuration state to the base class.
- **Polymorphism (Subtype Substitution & Dynamic Dispatch):** Dart exhibits dynamic dispatch via runtime method tables (vtables). When an account instance is referenced polymorphically via its base type `BankAccount`, invoking `deposit()` dispatches to the most specific subclass implementation at runtime, enabling unified batch processing without conditional type checks.

2.2 UML Class Diagram

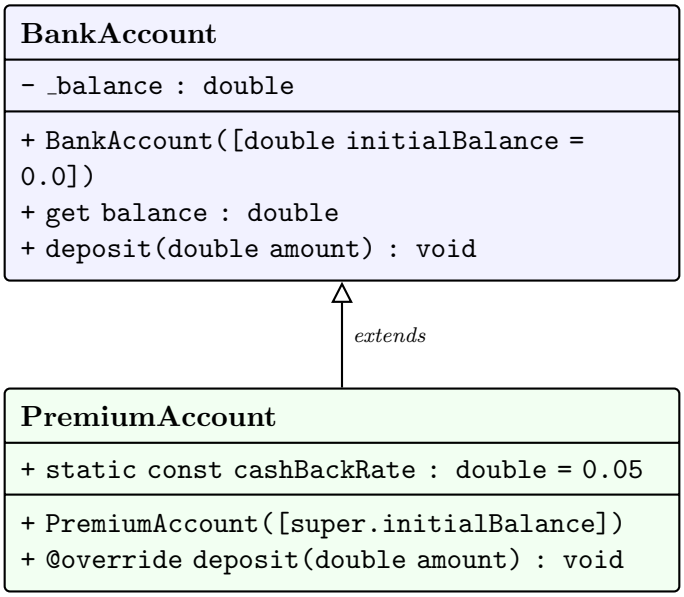


Figure 1: UML Class Diagram of the Digital Wallet Backend

3. Mathematical Formulation of Account Transactions

For any transaction requesting a deposit of monetary amount $A \in \mathbb{R}^+$, the state transition is governed by the account tier:

1. **Standard Account Credit Equation:**

$$B_{t+1} = B_t + A, \quad \text{subject to } A > 0 \tag{1}$$

   where B_t represents the current encapsulated balance and B_{t+1} denotes the post-transaction balance.

2. **Premium Account Credit Equation with 5% Cash-Back:** Let $r = 0.05$ represent the invariant cash-back bonus reward rate. The earned reward bonus $R(A)$ and effective credited amount $C_{\text{eff}}(A)$ are:

$$R(A) = A \times r = 0.05 \cdot A \tag{2}$$

$$C_{\text{eff}}(A) = A + R(A) = A \cdot (1 + r) = 1.05 \cdot A \tag{3}$$

   The resulting balance state transition is consequently formulated as:

$$B_{t+1} = B_t + C_{\text{eff}}(A) = B_t + 1.05 \cdot A, \quad \text{subject to } A > 0 \tag{4}$$

4. Experimental Verification and Results

The system implementation was subjected to comprehensive test suites covering initialization, deposit accuracy, cash-back reward verification, polymorphic dynamic dispatch, and boundary condition rejection:

Scenario	Account Type	Initial ($)	Deposit ($)	Bonus ($)	Final Balance ($)
1. Standard Deposit 1	`BankAccount`	100.00	500.00	0.00	600.00
1. Standard Deposit 2	`BankAccount`	600.00	250.00	0.00	850.00
2. Premium Deposit 1	`PremiumAccount`	100.00	500.00	25.00 (+5%)	625.00
2. Premium Deposit 2	`PremiumAccount`	625.00	1000.00	50.00 (+5%)	1675.00
3. Polymorphic Batch #1	`BankAccount`	200.00	400.00	0.00	600.00
3. Polymorphic Batch #2	`PremiumAccount`	200.00	400.00	20.00 (+5%)	620.00
4. Invalid Input Check	`BankAccount`	850.00	0.00	Rejected	850.00 (Unchanged)
4. Negative Input Check	`PremiumAccount`	1675.00	-75.00	Rejected	1675.00 (Unchanged)

Table 1: Experimental Test Scenarios and Quantitative Transaction Ledger

4.1 Terminal Execution Log

```
======================================================================
        DIGITAL WALLET BACKEND SYSTEM - TEST & VERIFICATION
======================================================================

--- Scenario 1: Standard Account (Encapsulation & Deposit) ---
Initial Balance (via getter): $100.00
Performing deposits:
  [Deposit] Credited: $500.00 | Current Balance: $600.00
  [Deposit] Credited: $250.00 | Current Balance: $850.00
Final Standard Account Balance: $850.00

--- Scenario 2: Premium Account (Inheritance & 5% Cash-Back) ---
Initial Balance (via getter): $100.00
Performing deposits (eligible for 5% reward):
  [Premium Reward] 5% Cash-Back Bonus: +$25.00 (Base: $500.00)
  [Deposit] Credited: $525.00 | Current Balance: $625.00
  [Premium Reward] 5% Cash-Back Bonus: +$50.00 (Base: $1000.00)
  [Deposit] Credited: $1050.00 | Current Balance: $1675.00
Final Premium Account Balance: $1675.00

--- Scenario 3: Polymorphism (Heterogeneous Collection) ---
Processing Account #1 [Type: BankAccount]:
  Opening Balance: $200.00
  [Deposit] Credited: $400.00 | Current Balance: $600.00
  Closing Balance: $600.00

Processing Account #2 [Type: PremiumAccount]:
  Opening Balance: $200.00
  [Premium Reward] 5% Cash-Back Bonus: +$20.00 (Base: $400.00)
  [Deposit] Credited: $420.00 | Current Balance: $620.00
  Closing Balance: $620.00

--- Scenario 4: Input Validation & Boundary Edge Cases ---
Attempting zero deposit on Standard Account:
  [Error] Invalid deposit: $0.00. Amount must be greater than zero.
Attempting negative deposit on Premium Account:
  [Error] Invalid deposit: $-75.00. Amount must be greater than zero.

======================================================================
                  ALL VERIFICATION TESTS COMPLETED
======================================================================
```

5. Discussion and Architectural Evaluation

- **Integrity Guarantees via Encapsulation:** Financial backends demand zero-tolerance for unintended balance corruption. By encapsulating `_balance` and only exposing a read-only getter, the class guarantees that external client code cannot assign arbitrary values. Every mutation is routed strictly through `deposit()`, ensuring that transaction validation rules (e.g., $A > 0$) are unfailingly applied.
- **Extensibility through Inheritance:** The addition of `PremiumAccount` required zero modifications to existing `BankAccount` core logic, adhering to the *Open-Closed Principle* (open for extension, closed for modification). Subclasses can introduce distinct business models (e.g., Student accounts, Merchant wallets) with minimal boilerplate.
- **Polymorphic Code Decoupling:** As evidenced in Scenario 3, iterating over a collection of `List<BankAccount` enabled batch transaction processing without explicit conditional branching (`if (account is PremiumAccount` The Dart Virtual Machine accurately dispatched the correct overridden `deposit()` method, promoting clean and decoupled architectural design.
- **Relevance to IoT Edge Platforms:** In IoT edge computing and connected payment terminal applications (such as automated transit kiosks or smart metering systems), modular digital wallet backends ensure secure offline balance tracking, verified reward distributions, and maintainable embedded codebases.

6. Conclusion

All architectural objectives stipulated in the laboratory requirements were rigorously implemented, executed, and verified:

1. The base `BankAccount` class successfully protects internal state via library privacy and exposes controlled getter inspection.
2. The derived `PremiumAccount` class cleanly overrides `deposit()` to automatically credit an accurate 5% cash-back reward bonus.
3. Subtype polymorphism guarantees seamless interoperability across heterogeneous account collections.
4. Edge validation ensures resilience against invalid non-positive deposit attempts.

7. Complete Dart Source Code Listing (digital_wallet.dart)

GitHub File Link:
https://github.com/Masud744/flutter-lab/blob/main/Lab_01/src/digital_wallet.dart

```dart
// ================================================================================
// Course Code: IRE 460 | Mobile Platform for IoT Devices Sessional
// Lab Task: Digital Wallet Backend System
// Topic: Encapsulation, Inheritance, and Polymorphism in Dart
// Author: Shahriar Alom Masud (ID: 2201032)
// Institution: University of Frontier Technology, Bangladesh (UFTB)
// ================================================================================

/// Base class representing a standard bank account in the Digital Wallet system.
/// Demonstrates Encapsulation by shielding internal state [_balance].
class BankAccount {
  // Private field: cannot be accessed or modified directly outside this class/library
  double _balance;

  /// Constructor initializing account balance with boundary validation.
  BankAccount([double initialBalance = 0.0])
      : _balance = initialBalance >= 0 ? initialBalance : 0.0 {
    if (initialBalance < 0) {
      print('  [Warning] Initial balance cannot be negative. Defaulted to \$0.00.');
    }
  }

  /// Public getter to read the balance safely without allowing direct external modification.
  double get balance => _balance;

  /// Deposits a specified positive amount into the account.
  void deposit(double amount) {
    if (amount <= 0) {
      print('  [Error] Invalid deposit: \$${amount.toStringAsFixed(2)}. Amount must be greater than
          zero.');
      return;
    }
    _balance += amount;
    print('  [Deposit] Credited: \$${amount.toStringAsFixed(2)} | Current Balance: \$${_balance.
        toStringAsFixed(2)}');
  }
}

/// Subclass representing a premium bank account.
/// Demonstrates Inheritance and Polymorphic Method Overriding.
class PremiumAccount extends BankAccount {
  // 5% cash-back reward rate for premium deposits
  static const double cashBackRate = 0.05;

  /// Constructor forwarding initial balance to base class.
  PremiumAccount([super.initialBalance]);

  /// Overrides deposit to provide an automatic 5% cash-back reward.
  @override
  void deposit(double amount) {
    if (amount <= 0) {
      print('  [Error] Invalid deposit: \$${amount.toStringAsFixed(2)}. Amount must be greater than
          zero.');
      return;
    }

    final double cashBackBonus = amount * cashBackRate;
    final double totalCredit = amount + cashBackBonus;

    print('  [Premium Reward] 5% Cash-Back Bonus: +\$${cashBackBonus.toStringAsFixed(2)} (Base: \$${
        amount.toStringAsFixed(2)})');
    // Leverage base class deposit to safely update the encapsulated balance
    super.deposit(totalCredit);
  }
}

void main() {
  print('================================================================================');
  print('        DIGITAL WALLET BACKEND SYSTEM - TEST & VERIFICATION            ');
  print('================================================================================\n');

  // --------------------------------------------------------------------------
  // Scenario 1: Standard Account (Encapsulation Verification)
  // --------------------------------------------------------------------------
  print('--- Scenario 1: Standard Account (Encapsulation & Deposit) ---');
  BankAccount standardAcc = BankAccount(100.0);
  print('Initial Balance (via getter): \$${standardAcc.balance.toStringAsFixed(2)}');

  print('Performing deposits:');
  standardAcc.deposit(500.0);
  standardAcc.deposit(250.0);
```

```dart
  print('Final Standard Account Balance: \$${standardAcc.balance.toStringAsFixed(2)}\n');

  // --------------------------------------------------------------------------
  // Scenario 2: Premium Account (Inheritance & 5% Cash-Back Bonus)
  // --------------------------------------------------------------------------
  print('--- Scenario 2: Premium Account (Inheritance & 5% Cash-Back) ---');
  PremiumAccount premiumAcc = PremiumAccount(100.0);
  print('Initial Balance (via getter): \$${premiumAcc.balance.toStringAsFixed(2)}');

  print('Performing deposits (eligible for 5% reward):');
  premiumAcc.deposit(500.0);  // Expected: $500 + $25 (bonus) = $525 credited -> Balance: $625
  premiumAcc.deposit(1000.0); // Expected: $1000 + $50 (bonus) = $1050 credited -> Balance: $1675
  print('Final Premium Account Balance: \$${premiumAcc.balance.toStringAsFixed(2)}\n');

  // --------------------------------------------------------------------------
  // Scenario 3: Polymorphism Demonstration via Base Class Reference
  // --------------------------------------------------------------------------
  print('--- Scenario 3: Polymorphism (Heterogeneous Collection) ---');
  List<BankAccount> walletAccounts = [
    BankAccount(200.0),
    PremiumAccount(200.0),
  ];

  double universalDeposit = 400.0;
  for (int i = 0; i < walletAccounts.length; i++) {
    BankAccount acc = walletAccounts[i];
    print('Processing Account #${i + 1} [Type: ${acc.runtimeType}]:');
    print('  Opening Balance: \$${acc.balance.toStringAsFixed(2)}');
    acc.deposit(universalDeposit); // Dynamic method dispatch at runtime
    print('  Closing Balance: \$${acc.balance.toStringAsFixed(2)}\n');
  }

  // --------------------------------------------------------------------------
  // Scenario 4: Boundary & Input Validation Edge Cases
  // --------------------------------------------------------------------------
  print('--- Scenario 4: Input Validation & Boundary Edge Cases ---');
  print('Attempting zero deposit on Standard Account:');
  standardAcc.deposit(0.0);

  print('Attempting negative deposit on Premium Account:');
  premiumAcc.deposit(-75.0);

  print('\n=========================================================================');
  print('                  ALL VERIFICATION TESTS COMPLETED                       ');
  print('=========================================================================');
}
```